Jabref System Architecture and Purpose

Daniel Gyorgy Szittya

March 3, 2026

JabRef is a citation and reference management tool written in Java. It helps manage bibliographic references stored in BibTeX and BibLaTeX formats. It has a graphical user interface built with JavaFX, a command-line interface (JabKit), and integrates with external tools such as LaTeX editors, Microsoft Word, and web browsers.

JabRef follows a layered architecture with clear separation of concerns. The codebase is organized into multiple Gradle subprojects, each with a different responsibility:

- jablib is the core library containing the model and logic packages that provide data structures and business logic.
- jabgui is the JavaFX-based graphical user interface, containing the gui package .
- jabkit is a command-line interface for batch operations.
- jabsrv is a server component for remote operations.
- build-logic contains Gradle build scripts and custom plugins for code generation and resource processing.

The core packages follow a strict dependency hierarchy:

```
gui --> logic --> model
gui ------------> preferences
cli --> logic --> model
```

- org.jabref.model contains the fundamental data structures with minimal logic. Key classes include BibDatabase (a collection of entries), BibEntry (a single bibliographic record with fields and citation key), and MetaData (library-level configuration). The model layer has no dependencies on other JabRef packages.
- org.jabref.logic implements business logic as an API that the GUI can invoke. This includes:
  - importer/exporter for eading and writing various bibliographic formats
  - cleanup for automated entry normalization and field transformations
  - journals for journal and conference abbreviation management
  - integrity for entry validation and consistency checking
  - citationkeypattern for citation key generation
- org.jabref.gui is the user interface layer built with JavaFX.
- preferences: Stores user-configurable settings, accessible primarily from the GUI layer.

JabRef uses an event bus (Google Guava EventBus) to decouple components. When the model changes, events are published and consumed by interested listeners in outer layers.

0.1 Abbreviation Subsystem (Issue #12728 focused here)

The abbreviation subsystem, located in .jabref.logic.journals, manages journal and conference name abbreviations. The architecture relevant to our issue includes:

- Abbreviation: A value object holding full name, abbreviated form, dotless form, and shortest unique abbreviation.
- JournalAbbreviationRepository: Stores and retrieves abbreviations from an MVStore database file. Supports exact and fuzzy matching.
- JournalAbbreviationLoader: Loads abbreviations from bundled resources and external CSV files.
- JournalAbbreviationPreferences: User preferences for abbreviation behavior.
- AbbreviateJournalCleanup / UnabbreviateJournalCleanup: Cleanup jobs that apply abbreviations to journal and journaltitle fields.

Our refactoring extends this subsystem to also handle conference proceedings by introducing a parallel ConferenceAbbreviationRepository and generalizing shared code into base classes.

Figure 1 illustrates the layered architecture and the key classes involved in the abbreviation feature.

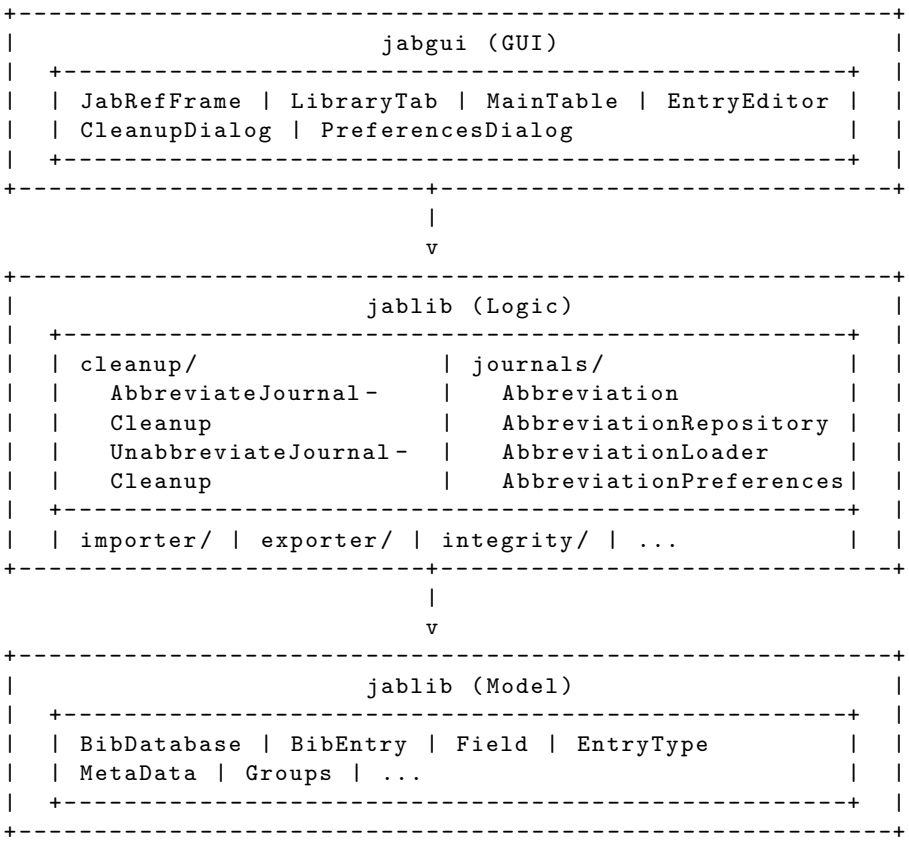

Figure 1: JabRef Layered Architecture with Abbreviation Subsystem